A Summer Internship - I Project Report on

"SPENDWISE: PERSONAL EXPENDITURE MANAGEMENT SYSTEM"

Submitted in partial fulfilment of the Requirements for the award of the Degree of

Bachelor of Technology

in

Computer Science & Engineering

By

ANUMANDLA SNITHESH
24EG105R38

Under the Guidance of

_____

Department of Computer Science & Engineering

ANURAG UNIVERSITY

GHATKESAR (M), MEDCHAL DISTRICT,

Hyderabad - 500088

Academic Year: 2026-2027

ANURAG UNIVERSITY

Department of Computer Science & Engineering

CERTIFICATE

I, ANUMANDLA SNITHESH, bearing hall ticket number 24EG105R38, hereby declare that the project report entitled "SPENDWISE: PERSONAL EXPENDITURE MANAGEMENT SYSTEM", submitted to the Department of Computer Science & Engineering, Anurag University, Hyderabad, is a record of bonafide work carried out by me in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Signature of The Guide

_____

ANUMANDLA SNITHESH
24EG105R38

ACKNOWLEDGMENT

I would like to express my sincere thanks and deep sense of gratitude to my project supervisor _____, for his/her constant encouragement and inspiring guidance without which this project could not have been completed. His/her critical reviews, valuable insights, and constructive feedback significantly enhanced my understanding of full-stack engineering and steered the project to fruitful completion.

I would like to express my sincere gratitude to Dr. Archana Mantri, Vice Chancellor, Anurag University, and Dr. V. Vijaya Kumar, Dean, School of Engineering, Anurag University, for their continuous encouragement, vision, and administrative support throughout the academic tenure.

I would also like to acknowledge my sincere gratitude for the support extended by Dr. P. Raja Sekhar Reddy, Head of the Department, CSE Cluster-1, whose commitment to academic excellence and relentless motivation provided the right environment during the crucial stages of our project work.

Finally, I express my heartfelt gratitude to my family, faculty members, and peers who supported and encouraged me directly and indirectly during the development of this project.

ANUMANDLA SNITHESH 24EG105R38 Department of CSE, Anurag University

DECLARATION

I hereby declare that the project entitled “SPENDWISE: PERSONAL EXPENDITURE MANAGEMENT SYSTEM”, submitted to the Department of Computer Science & Engineering, Anurag University, Hyderabad, is an authentic record of the original work carried out by me under the guidance of _____.

I further declare that this report represents my own individual effort and has not been submitted previously to any other university, college, or examination body for the conferment of any degree, diploma, fellowship, or other recognition.

ANUMANDLA SNITHESH 24EG105R38

TABLE OF CONTENTS

TOPIC	SECTION
Acknowledgement	i
Declaration	ii
Abstract	iii
1. Introduction	1
2. Problem Statement	2
3. Objectives	3
4. Scope of the Project	4
5. Existing System	5
6. Proposed System	6
7. Software & Hardware Requirements	7
8. System Design	8
9. Flowchart & Algorithm	9
10. Modules Description	10
11. Implementation	11
12. Results / Output Screens	12
13. Advantages & Limitations	13
14. Future Enhancements	14
15. Conclusion	15
16. References	16
17. Source Code & Architecture	17
Project Summary	18

ABSTRACT

SpendWise is an enterprise-grade, cloud-native personal expenditure management web application engineered to revolutionize personal financial planning, transaction tracking, budget adherence, and savings monitoring for individuals and households. In the fast-paced modern economic environment, individuals encounter numerous challenges in tracking daily discretionary spending, managing recurring obligations, adhering to monthly category allocations, and maintaining disciplined savings habits when relying on manual spreadsheets, paper notebooks, or fragmented notes.

To resolve these challenges, SpendWise introduces a robust, modern decoupled architecture. The frontend is engineered using React 18, Vite, TypeScript, and Tailwind CSS, providing an ultra-responsive, accessible user interface with interactive financial visualizations. The backend is built upon Java 21 and the Spring Boot 3.3 framework, adopting clean architectural patterns, Spring Security 6 with stateless JSON Web Token (JWT) authentication, and Spring Data JPA backed by a high-performance database schema (MySQL 8 with automated Flyway database migrations and an instant in-memory H2 MySQL-compatibility profile for zero-configuration deployments).

SpendWise provides comprehensive end-to-end functionality including secure user authentication, interactive dashboard metrics, categorized transaction management (income vs. expense), real-time budget utilization tracking with visual threshold alerts, goal-oriented savings progress management, and one-click export of financial statements into standard CSV format. Deployed via Docker containers on Render cloud infrastructure, SpendWise guarantees high availability, sub-second latency, bank-grade credential security (BCrypt hashing), and complete multi-device responsiveness, establishing a dependable digital ecosystem for personal wealth stewardship.

1. INTRODUCTION

SpendWise (Personal Expenditure Management System) is a comprehensive web-based application designed to digitalize, streamline, and modernize the manner in which individuals record, analyze, and manage their personal finances. In today's digital age, economic transactions occur rapidly across multiple channels—cash, debit/credit cards, UPI, digital wallets, and net banking. Consequently, maintaining an accurate, consolidated overview of cash flow has become an increasingly daunting responsibility for students, working professionals, and households.

Traditionally, individuals have attempted to monitor their personal spending through physical diary entries, paper receipts, or static spreadsheets like Microsoft Excel. While these manual methods offer initial familiarity, they suffer from significant drawbacks: they are tedious, time-consuming, highly susceptible to calculation errors, incapable of providing proactive budget breach alerts, and vulnerable to catastrophic data loss due to physical damage or file corruption. Moreover, traditional methods lack real-time visual insights, leaving users unaware of subtle "cash leaks" and impulsive spending patterns until their monthly capital is depleted.

With the rapid evolution of cloud computing and web technologies, there is an imperative need for an intuitive, secure, and automated personal finance platform. SpendWise answers this need by delivering a lightweight yet technologically robust full-stack solution. The front-end leverages React, Vite, and Tailwind CSS to guarantee blistering rendering speeds, modern aesthetic appeal, and seamless responsiveness across desktops, laptops, tablets, and smartphones. The back-end is powered by enterprise-grade Java 21 and Spring Boot 3, providing rock-solid RESTful endpoints, bulletproof stateless JWT authentication, and resilient data persistence.

Through SpendWise, users gain complete visibility and control over their financial journey. The application empowers users to register secure personal profiles, record detailed income and expense transactions, categorize spending into essential and discretionary buckets, set monthly budgetary caps with real-time progress indicators, define structured savings milestones, and export comprehensive statements. In summary, SpendWise bridges the gap between intricate financial management and effortless daily usability, fostering disciplined financial habits and long-term economic well-being.

2. PROBLEM STATEMENT

Effective personal financial management is essential for financial independence and peace of mind. However, the majority of individuals struggle with financial tracking due to the inherent shortcomings of conventional and manual methodologies:

- High Propensity for Human Calculation Errors: Manual arithmetic in paper registers or spreadsheets frequently results in erroneous balances, miscategorized transactions, and conflicting monthly totals.
- Time Consumption and Disorganization: Recording transactions by hand at the end of every day or week is cumbersome. Over time, physical notebooks become disorganized, damaged, or lost, making retrospective audits practically impossible.
- Absence of Proactive Budget Alerts: Static spreadsheets cannot notify a user when their dining, entertainment, or shopping expenditures are nearing or exceeding their monthly limit.
- Lack of Intuitive Visual Analytics: Raw tables of numbers fail to clearly convey spending trends, income-to-expense ratios, and category distributions, preventing users from making timely corrective financial decisions.
- Security and Privacy Vulnerabilities: Physical notebooks or unencrypted local spreadsheets leave sensitive personal financial data exposed to unauthorized viewing, accidental deletion, or hardware crashes.
- Generic, Ad-Riddled Existing Apps: Many commercial mobile applications require intrusive banking SMS permissions, bombard users with targeted loan advertisements, or lock essential budgeting features behind expensive recurring subscriptions.

Therefore, there is an urgent demand for a dedicated, secure, user-centric, and automated Personal Expenditure Management System that guarantees complete data ownership, real-time analytics, automated budget monitoring, and seamless cloud accessibility without intrusive tracking.

3. OBJECTIVES

The primary objective of SpendWise is to conceptualize, engineer, and deploy an automated, reliable, and user-friendly web application that simplifies personal expense tracking, optimizes budget allocation, and cultivates healthy financial habits. The specific objectives include:

- Automated Transaction Tracking: Provide a swift, intuitive interface for logging income and expense entries with amount, category, date, payment method, and descriptive notes.

- Robust User Authentication and Privacy: Implement industry-standard security utilizing BCrypt password hashing and stateless JSON Web Tokens (JWT) to safeguard confidential user profiles.
- Dynamic Budget Monitoring: Allow users to define monthly category budgets (e.g., Groceries, Utilities, Entertainment) and track spending against these limits in real-time with visual warning thresholds.
- Goal-Oriented Savings Management: Enable users to establish specific savings targets (e.g., Emergency Fund, Vacation, New Laptop) and log incremental deposits toward those goals.
- Comprehensive Analytics Dashboard: Render instant high-level summaries including Total Balance, Monthly Income, Monthly Expenses, Savings Rate, and visual breakdown charts.
- Export and Reporting Capabilities: Implement automated data serialization to allow users to download their complete transaction logs in standard CSV format for external auditing or tax preparation.
- Cross-Device Responsiveness and Cloud Deployment: Build a responsive interface with Tailwind CSS and package the backend into a containerized Docker image deployed on Render cloud infrastructure with zero operational barriers.

4. SCOPE OF THE PROJECT

The scope of SpendWise encompasses the end-to-end lifecycle of personal financial tracking, spanning user acquisition to structured analytical reporting. The operational boundaries include:

- User Domain: Individual users, students, freelancers, and household managers seeking an ad-free, secure platform to track cash flow and avoid debt traps.
- Functional Scope: User registration, secure login, password management, profile customization, dynamic category management, transaction CRUD operations, budget tracking, savings goal management, and data export.
- Architectural Scope: A decoupled client-server architecture where the frontend (React Vite SPA) communicates with the backend (Java Spring Boot) exclusively via secure RESTful JSON APIs.
- Deployment Scope: Automated deployment pipelines utilizing containerized Docker runtimes on Render cloud, supporting cloud-hosted MySQL databases as well as zero-setup in-memory database profiles for seamless demonstrations.
- Scalability Scope: While optimized for individual single-user tenancy per account, the backend architecture supports multi-tenant isolation, allowing thousands of simultaneous users to manage their data with strict row-level security.

5. EXISTING SYSTEM

The prevailing approaches to personal finance management typically fall into three traditional categories:

1. Manual Paper Registers / Physical Passbooks

Users write receipts and expenditures by hand into diaries. This system is completely manual, lacks search functionality, provides zero analytical charts, and is vulnerable to theft, moisture, and fire damage.

2. Desktop Spreadsheets (Microsoft Excel / Google Sheets)

Users manually maintain tabular spreadsheets. While formulas can compute totals, creating dynamic charts requires advanced spreadsheet skills. Furthermore, updating spreadsheets from mobile devices is clumsy, and there are no automated budget notifications.

3. Proprietary Commercial Expense Apps

Many commercial mobile apps read personal SMS messages without user comprehension, upload financial habits to marketing servers, lock multi-category features behind paywalls, or shut down unpredictably.

Limitations of Existing Systems: High manual effort, risk of data loss, privacy invasion, lack of customizability, absence of real-time alerts, and poor mobile-to-desktop cross-compatibility.

6. PROPOSED SYSTEM

SpendWise replaces the fragile, manual paradigms of existing systems with an automated, responsive, cloud-accessible, and private financial command center.

In SpendWise, every user interacts with a secure, beautifully crafted user interface. Transactions are committed to a relational database within milliseconds. Real-time calculations provide immediate updates to financial ratios, budget consumption meters, and savings goal progress.

Key Features of the Proposed System:

- Centralized Cloud Storage: Financial records are persisted securely and accessible anywhere across any device via standard web browsers.
- Stateless JWT Authentication: Zero session memory leaks on servers; tokens expire securely and refresh without user disruption.
- Visual Feedback & Gamification: Progress bars transition from green to amber and red as budgets reach 75% and 90% utilization, giving immediate psychological cues against overspending.
- One-Click Demo Autofill: Embedded demo capabilities pre-seed realistic multi-month data, allowing recruiters, evaluators, and prospective users to experience the application instantly.
- Audit-Ready Data Export: Export complete transaction histories into standard CSV formats in a single click.

7. SOFTWARE & HARDWARE REQUIREMENTS

To ensure reproducible builds, high availability, and rapid development, SpendWise is engineered using modern software stacks and lightweight hardware configurations.

Software Requirements

Component	Specification / Technology
Operating System	Windows 10/11, macOS, or Ubuntu Linux (Docker-compatible)
Backend Runtime	Java Development Kit (JDK) 21 (LTS)
Backend Framework	Spring Boot 3.3.4 (Web, Data JPA, Security, Actuator, Validation)
Security & Auth	Spring Security 6, JJWT 0.12.6 (HMAC-SHA256), BCrypt
Database	MySQL 8.0 (Production) / H2 In-Memory DB (Demo Profile)
Database Migration	Flyway 10.x Database Migration Engine
Frontend Runtime	Node.js v20.x or v24.x & npm v10.x
Frontend Framework	React 18 with TypeScript 5.x
Build Tool & Bundler	Vite 8.x / 5.x with Hot Module Replacement (HMR)
Styling & UI	Tailwind CSS, Lucide React Icons, React Hot Toast
Form Management	React Hook Form, Zod Validation Schema
HTTP Client	Axios 1.7.x with Custom Request & Response Interceptors
Cloud Platform	Render (Docker Web Service for Backend + Static Site for Frontend)
IDE & Version Control	VS Code / IntelliJ IDEA, Git & GitHub

Hardware Requirements

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (Intel Core i5 recommended for concurrent compilation).
- System Memory (RAM): Minimum 4 GB (8 GB or higher recommended for running Spring Boot JVM and Vite dev server simultaneously).
- Storage: 256 GB SSD or higher with at least 5 GB available free disk space.
- Network: High-speed broadband internet connectivity for cloud database synchronization and Render deployment.
- Display: Minimum screen resolution of 1366x768 (Full HD 1920x1080 recommended).

8. SYSTEM DESIGN

SpendWise adopts a tiered, decoupled Clean Architecture pattern that separates concerns into Presentation, Application/Service, and Data Persistence layers.

Architectural Overview

- Presentation Tier (Client): Single Page Application built in React. Encapsulates state using React Context API (`AuthContext`), manages client routing via React Router DOM 6, and executes authenticated asynchronous HTTP requests via Axios.

- API Gateway / Security Filter: Spring Security interceptor chain inspecting HTTP `Authorization: Bearer <JWT>` headers on all protected `/api/**` routes. Unauthenticated requests are rejected with standardized RFC 7807 JSON error responses.
- Service Tier: Business logic components (`TransactionService`, `BudgetService`, `SavingsService`, `ReportService`, `AuthService`) carrying out validations, calculations, and transactional boundaries (`@Transactional`).
- Data Tier: Spring Data JPA repositories interfacing with Hibernate ORM and PostgreSQL/MySQL dialect engines to execute optimized queries with row-level security bound to the authenticated user ID.

Entity-Relationship Design (Key Entities)

- User: Stores account credentials (`email`, `password_hash`, `name`, `currency`, `created_at`).
- Category: Defines spending/earning classifications (`name`, `type: INCOME/EXPENSE`, `color_code`, `icon`, `user_id`). Supports default system categories as well as user-customized categories.
- Transaction: Financial entries (`amount`, `type`, `date`, `description`, `payment_method`, `category_id`, `user_id`).
- Budget: Monthly category spending caps (`month`, `year`, `allocated_amount`, `category_id`, `user_id`).
- SavingsGoal: Long-term asset accumulation targets (`title`, `target_amount`, `current_amount`, `target_date`, `user_id`).
- SavingsGoalEntry: Individual deposit logs towards a specific savings goal.

9. FLOWCHART & ALGORITHM

The operation of SpendWise follows strictly orchestrated algorithms ensuring data consistency, mathematical accuracy, and cryptographic security.

Algorithm 1: User Authentication & JWT Issuance

Step 1: User submits login form with email and plaintext password.

Step 2: Client performs local Zod schema validation (checks valid email format and non-empty password).

Step 3: Client dispatches POST request to `/api/auth/login`.

Step 4: Server queries `UserRepository` for matching email. If not found, return HTTP 401 Unauthorized.

Step 5: Server invokes `PasswordEncoder.matches(plaintext, storedHash)` using BCrypt. If false, return HTTP 401.

Step 6: Server generates HMAC-SHA256 signed JWT containing Subject (User ID), IssuedAt, and Expiration (24 hours).

Step 7: Server returns HTTP 200 with JWT and user metadata. Client caches JWT in `localStorage` and redirects to `/dashboard`.

Algorithm 2: Transaction Creation & Real-Time Balance Update

Step 1: User clicks "Add Transaction" and completes amount, category, date, and type (Income/Expense).

Step 2: Axios attaches `Bearer <token>` and posts payload to `/api/transactions`.

Step 3: Spring Boot extracts user ID from SecurityContext token.

Step 4: Validates category belongs to the user or is a global default. Validates amount > 0.

Step 5: Persists transaction entity inside database transaction.

Step 6: Returns updated transaction object. Frontend state updates automatically and recomputes total balance and monthly totals without full page reload.

Algorithm 3: Budget Utilization & Warning Threshold Engine

Step 1: When user navigates to `/budgets`, backend receives request for current month and year.

Step 2: Query fetches all budgets allocated by the user for month M and year Y.

Step 3: For each budget category, query computes sum of all expense transactions in category C within month M: `ActualSpent = SUM(amount)`.

Step 4: Compute `PercentageUsed = (ActualSpent / AllocatedAmount) * 100`.

Step 5: Classify status: If Percentage < 75% -> "Normal" (Green); If 75% <= Percentage < 90% -> "Warning" (Amber); If Percentage >= 90% -> "Critical / Exceeded" (Red).

Step 6: Return DTO to frontend; UI renders animated progress bars with contextual warnings.

10. MODULES DESCRIPTION

SpendWise is structured into modular, loosely coupled components across both client and server layers:

1. Authentication & User Profile Module

Handles registration, credential validation, session establishment, and profile updates. Equipped with a one-click "Fill Demo" feature that allows users to instantly test the platform using pre-seeded test data (`demo@spendwise.com` / `Demo@123`).

2. Analytics Dashboard Module

The central control center. Computes live KPIs including Net Balance, Monthly Income, Monthly Outflow, and Savings Rate. Displays recent transactions and dynamic progress meters summarizing category budgets.

3. Transaction Management Module

Provides complete CRUD capabilities for financial transactions. Features multi-criteria filtering by date range, category, transaction type, and text search across merchant descriptions, with pagination support.

4. Category Management Module

Allows users to organize expenditures into sensible classifications (e.g., Housing, Food, Transportation, Entertainment, Health). Users can customize custom colors and icons for instant visual recognition.

5. Budget Tracking & Allocation Module

Enables setting proactive financial boundaries per category for each billing cycle. Visualizes allocated vs. actual consumption with color-coded warning badges.

6. Savings Goals Module

Facilitates goal-based financial discipline. Users configure targets, due dates, and initial balances, logging milestone contributions that dynamically increment target completion percentages.

7. Reports & CSV Export Module

Generates periodic cash-flow summaries and powers client-side and server-side CSV statement exports using standard Apache Commons CSV serialization, ensuring full portability of personal records.

11. IMPLEMENTATION

The implementation phase transformed architectural designs into an enterprise-ready, robust full-stack software system.

Backend Implementation Details

The backend is built upon Java 21 LTS leveraging Spring Boot 3.3.4. Key highlights include:

- Controller Layer: Exposes RESTful endpoints (`AuthController`, `TransactionController`, `BudgetController`, `SavingsGoalController`, `ReportController`) configured with `@Valid` request body bindings.
- Security Configuration (`SecurityConfig.java`): Configures stateless CSRF-disabled security filter chains. Replaces deprecated origin configurations with `setAllowedOriginPatterns("*")` to support credentials across dynamic cloud URLs on Render.
- JWT Filter (`JwtAuthenticationFilter.java`): Intercepts incoming requests, parses `Authorization` header, extracts username claim, and sets `UsernamePasswordAuthenticationToken` in Spring SecurityContext.
- Dual-Profile Database Setup (`application.yml`): Default profile targets production MySQL databases via HikariCP. A dedicated `demo` profile activates an in-memory H2 database running in MySQL-compatibility mode (`MODE=MySQL`) with Flyway migrations bypassed, enabling 15-second cold-starts on cloud free tiers.
- Memory & Heap Optimization (`Dockerfile`): Docker image based on lightweight Alpine JRE (`eclipse-temurin:21-jre-alpine`). Configured with JVM flags `-Xmx350m -Xms128m -XX:+UseG1GC` specifically engineered to thrive within Render's 512MB RAM free-tier ceiling.

Frontend Implementation Details

The client application is built with React 18 and Vite in TypeScript, delivering instantaneous response times:

- Custom API Client (`api.ts`): Configured with smart multi-tier backend auto-discovery. Dynamically pairs with onrender.com hostnames, reads `localStorage` overrides, or falls back to build-time environment variables.
- Authentication Interceptor: Automatically injects cached JWT tokens into all outgoing Axios requests and intercepts HTTP 401 responses to clear expired credentials and redirect to login.

- Component Architecture: Modular component hierarchy separating reusable atomics (`Button`, `Input`, `Card`, `Badge`, `Modal`) from domain pages (`DashboardPage`, `TransactionsPage`, `BudgetsPage`, `SavingsGoalsPage`).

12. RESULTS / OUTPUT SCREENS

SpendWise was thoroughly tested across multiple browser environments (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari Mobile). All functional requirements performed flawlessly.

1. Public Landing Page

Presents a modern, welcoming introduction to the SpendWise platform. Highlights key value propositions (real-time budgeting, savings milestones, complete data privacy) with quick action buttons directing to Registration and Sign In.

2. Authentication Screen with Demo Autofill

Secure Sign In interface featuring email and password fields with toggleable visibility. Includes a dedicated "Fill Demo" button that instantly loads `demo@spendwise.com` and pre-seeded multi-month data for one-click testing.

3. Analytics & Financial Dashboard

Displays real-time financial health cards: Total Balance, Monthly Inflow, Monthly Outflow, and Net Savings. Features summary tables of recent expenditures and category budget status bars.

4. Transaction Management & Filter Screen

Tabular view of all historical entries. Users can filter by Date, Category, or Type (Income/Expense), conduct instant keyword searches, and add/edit/delete transactions with responsive modal dialogs.

5. Budget Allocation & Monitoring Screen

Displays categorized monthly budget progress bars. Color cues provide immediate visual feedback: Green indicates safe spending, Orange warns of approaching limits (75%+), and Red signals budget overruns.

6. Savings Goals & Milestone Tracker

Visualizes long-term financial targets with target dates and circular/linear percentage meters. Users can click "Deposit" to log incremental savings contributions in real-time.

7. CSV Statement Export Output

A single click on "Export CSV" downloads a structured, timestamped comma-separated spreadsheet file containing date, category, description, payment method, type, and exact amounts.

13. ADVANTAGES & LIMITATIONS

Advantages

- Zero Manual Calculation Errors: All balances, monthly totals, category aggregations, and savings percentages are calculated instantaneously by robust backend algorithms.
- Strict Data Privacy & Ownership: No third-party ad networks, no automated bank SMS scraping, and no commercial data reselling. The user retains absolute control over their data.
- Enterprise Security: Passwords are protected using BCrypt hashing with salted iterations; all protected REST endpoints enforce stateless JWT cryptographic verification.
- Cross-Device Accessibility: Responsive Tailwind CSS layouts ensure the application renders cleanly across desktops, tablets, and mobile smartphones.
- High Performance: Lightweight Vite bundle and optimized Alpine Docker container result in lightning-fast initial load times and snappy user interactions.
- Zero-Friction Demonstrations: Pre-packaged in-memory database profile and frontend demo button allow evaluators to test all features in seconds without setting up external MySQL instances.

Limitations

- Cloud Free-Tier Cold Starts: On free hosting tiers like Render, inactive backend containers spin down after 15 minutes of inactivity, resulting in a 25–35 second latency on the initial wake-up request.
- No Direct Bank API Integration: Transactions must be logged manually or via file upload, as direct open-banking API integration (e.g., Plaid/Salt Edge) requires licensed financial institution partnerships.
- Single Currency per User: The current version does not support multi-currency real-time exchange rate conversions within a single portfolio.

14. FUTURE ENHANCEMENTS

SpendWise offers an extensible foundation for several high-impact future enhancements:

- AI-Powered Receipt OCR Scanning: Integrating Optical Character Recognition (OCR) using Tesseract or Google Cloud Vision to allow users to photograph paper receipts and automatically parse date, merchant, and line items.

- Predictive Expense Forecasting: Implementing machine learning algorithms (e.g., ARIMA or linear regression) to analyze historical spending patterns and project end-of-month cash balances.
- Open Banking Integration: Integrating Account Aggregator / Plaid APIs to enable optional automated bank transaction synchronization for verified user accounts.
- Native Mobile Applications: Packaging the React application into React Native or Flutter to provide native push notifications when category budgets reach 85% utilization.
- Collaborative Household Budgets: Introducing multi-user shared family wallets with role-based permissions (Owner, Contributor, Viewer).

15. CONCLUSION

The SpendWise Personal Expenditure Management System successfully modernizes and digitalizes personal financial management, replacing error-prone paper notebooks and clumsy spreadsheets with an automated, secure, and responsive web platform.

By harmonizing React 18, Vite, TypeScript, Tailwind CSS, Java 21, Spring Boot 3, Spring Security, and Docker cloud deployment on Render, SpendWise demonstrates how contemporary software engineering principles can be combined to solve real-world personal challenges. The application fulfills all technical and operational objectives, delivering secure authentication, categorized transaction tracking, proactive budget monitoring, savings goal visualization, and instant data export.

In conclusion, SpendWise empowers users to take disciplined control of their financial destiny, minimize discretionary cash waste, and attain long-term economic peace of mind. The project provided invaluable practical experience in full-stack architecture, API design, containerization, and modern cloud deployment standards.

16. REFERENCES

The following technical documentation, specifications, and books were referenced during the design and engineering of SpendWise:

- 1. Spring Boot Official Documentation. Pivotal / VMware Tanzu. https://docs.spring.io/spring-boot/docs/current/reference/html/
- 2. Spring Security 6 Reference Architecture. https://docs.spring.io/spring-security/reference/
- 3. React 18 Documentation & Core Hooks Guide. Meta Open Source. https://react.dev
- 4. Vite: Next Generation Frontend Tooling. https://vitejs.dev
- 5. JSON Web Token (JWT) RFC 7519 Specification. Internet Engineering Task Force (IETF). https://tools.ietf.org/html/rfc7519
- 6. Tailwind CSS Framework Documentation. Tailwind Labs. https://tailwindcss.com/docs
- 7. Docker Engine & Multi-Stage Builds Documentation. Docker Inc. https://docs.docker.com/build/building/multi-stage/
- 8. Render Cloud Deployment Guides & Blueprint Specifications. https://render.com/docs
- 9. Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- 10. Walls, Craig. Spring in Action, Sixth Edition. Manning Publications, 2022.

17. SOURCE CODE & REPOSITORY STRUCTURE

SpendWise is organized in a clean, decoupled monorepo architecture with distinct separation between frontend client assets and backend microservice components.

Project Folder Structure

```
SpendWise-PersonalExpenditure/
├── backend/
│   ├── pom.xml                                # Maven Project Descriptor (Java 21, Spring Boot 3.3.4)
│   ├── Dockerfile                             # Multi-stage optimized Alpine JRE build
│   └── src/main/
│       ├── java/com/spendwise/
```

```
│       │   ├── SpendwiseBackendApplication.java
│       │   ├── config/                        # SecurityConfig,
JwtAuthenticationFilter, WebConfig
│       │   ├── controller/                    # Auth, Transaction, Budget,
Savings, Report Controllers
│       │   ├── dto/                           # Request/Response Data Transfer
Objects
│       │   ├── entity/                        # User, Transaction, Category,
Budget, SavingsGoal JPA Entities
│       │   ├── repository/                    # Spring Data JPA Repository
Interfaces
│       │   └── service/                       # Business Logic &
DataSeedService
│       └── resources/
│           ├── application.yml                # Multi-profile configuration
(MySQL & Demo H2)
│           └── db/migration/                  # Flyway SQL Database Migrations
├── frontend/
│   ├── package.json                           # NPM dependencies (React 18,
Vite, Tailwind, Axios)
│   ├── vite.config.ts                         # Vite compilation & dev proxy
configuration
│   ├── tailwind.config.js                     # Custom Tailwind design tokens
& brand colors
│   └── src/
│       ├── App.tsx                            # Main React application router
wrapper
│       ├── main.tsx                           # DOM bootstrap root
│       ├── components/                        # Common UI widgets (Button,
Input, Card, Modal, Navbar, Sidebar)
│       ├── context/                           # AuthContext (JWT state
management)
│       ├── pages/                             # Auth, Dashboard, Transactions,
Budgets, Savings, Reports
│       ├── services/                          # Axios client (api.ts) & domain
service abstractions
│       └── types/                             # TypeScript interfaces & domain
models
├── render.yaml                                # Render Blueprint
Infrastructure-as-Code
└── README.md                                  # Architecture, setup &
deployment guide
```

Key Backend Controller Implementation (TransactionController.java)

```
@RestController
```

```
@RequestMapping("/api/transactions")
@RequiredArgsConstructor
public class TransactionController {
    private final TransactionService transactionService;

    @GetMapping
    public ResponseEntity<Page<TransactionResponse>> getAllTransactions(
            @AuthenticationPrincipal Long userId,
            @RequestParam(required = false) TransactionType type,
            @RequestParam(required = false) Long categoryId,
            @PageableDefault(sort = "date", direction = Sort.Direction.DESC)
Pageable pageable) {
        return ResponseEntity.ok(transactionService.getTransactions(userId,
type, categoryId, pageable));
    }

    @PostMapping
    public ResponseEntity<TransactionResponse> createTransaction(
            @AuthenticationPrincipal Long userId,
            @Valid @RequestBody TransactionRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(transactionService.createTransaction(userId, request));
    }
}
```

Key Frontend API Service Implementation (api.ts)

```
export const getApiBaseUrl = (): string => {
  const customUrl = localStorage.getItem("spendwise_api_url");
  if (customUrl?.trim()) return customUrl.trim();
  if (window.location.hostname.endsWith(".onrender.com")) {
    return `https://${window.location.hostname.replace("spendwise-web",
"spendwise-api")}/api`;
  }
  return "/api";
};
export const api = axios.create({ baseURL: getApiBaseUrl() });
```

PROJECT SUMMARY

The SpendWise Personal Expenditure Management System was successfully designed, developed, containerized, and deployed as an enterprise-grade full-stack web application. The

project achieved all targeted milestones across every phase of software engineering—from comprehensive requirement elicitation and relational database modeling to modular REST API development, responsive component creation, and cloud orchestration.

By replacing antiquated paper registers and fragile spreadsheets with an automated, responsive digital platform, SpendWise empowers users with accurate financial awareness, proactive budget safeguards, and disciplined savings motivation. The project demonstrated the formidable capabilities of modern web technologies—combining the reactivity of React 18 and Vite with the rock-solid dependability of Java 21 Spring Boot and Docker.

The practical knowledge acquired throughout the design and realization of SpendWise has immensely fortified our understanding of full-stack system architecture, stateless JWT security paradigms, containerization best practices, and agile cloud deployment methodologies.